Machine Learning for Business Analytics

GTSC2143

Lecture 1 Introduction

Instructor: Dr. Lily Bei Luo

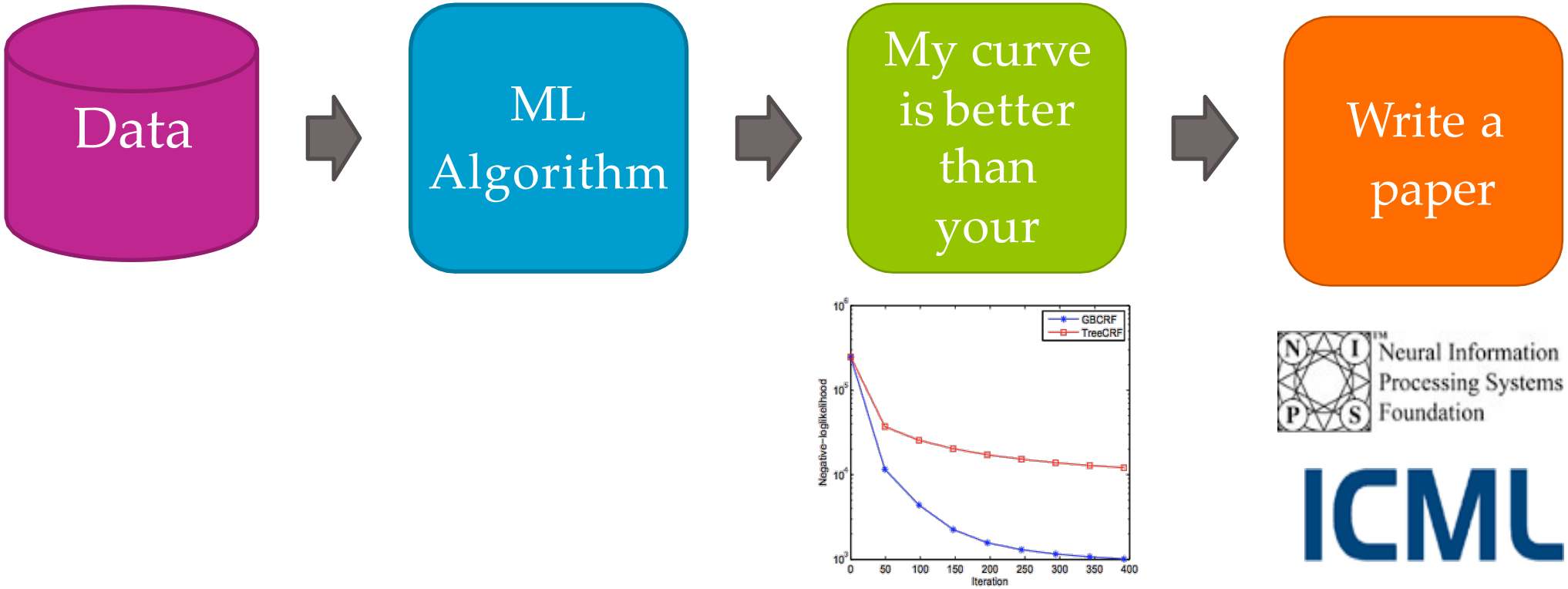
AY2025-2026 Semester 1

北京师范大学 香港浸会大学 联合国际学院
BEIJING NORMAL UNIVERSITY · HONG KONG BAPTIST UNIVERSITY
UNITED INTERNATIONAL COLLEGE

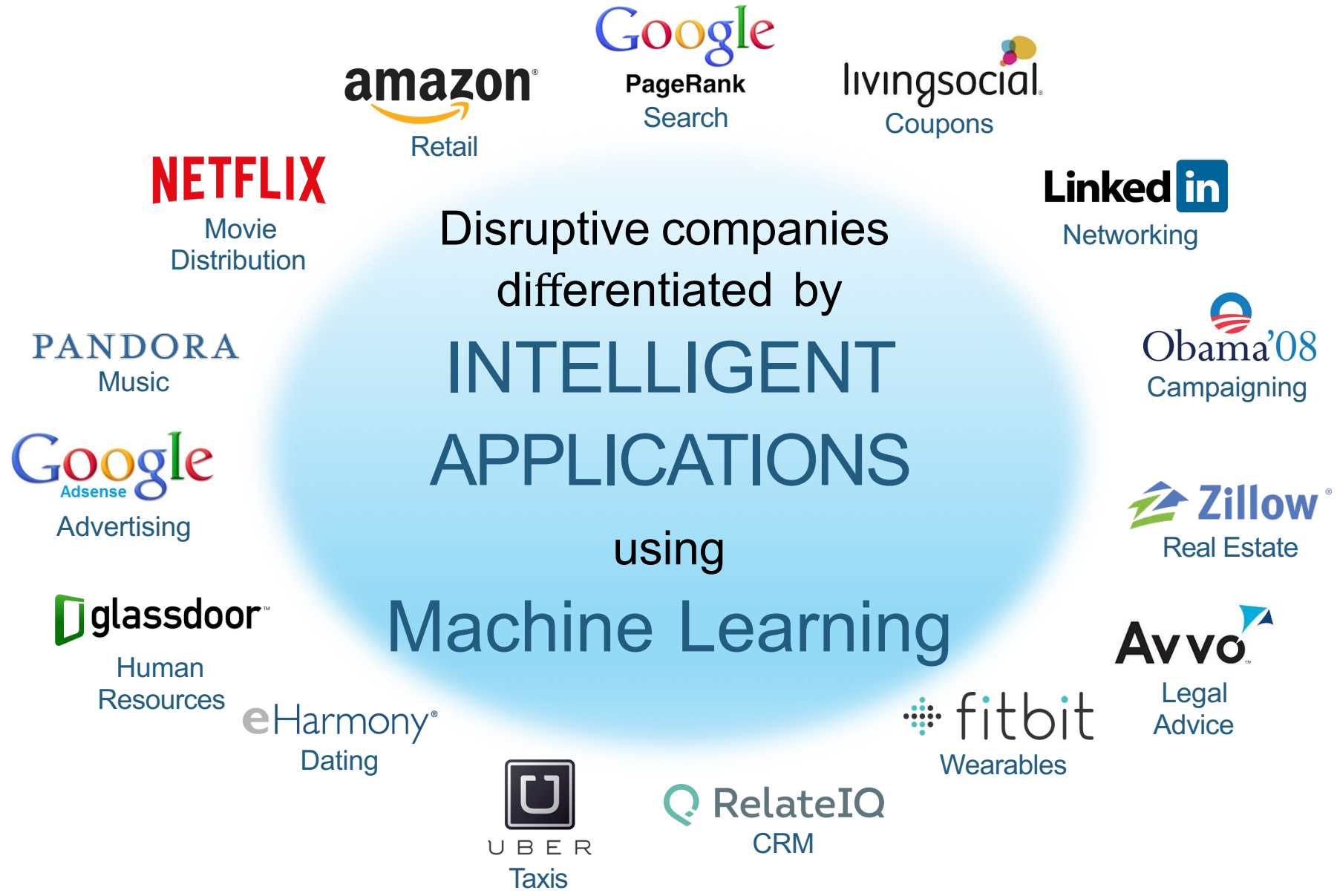
Section 1 Introduction to Machine Learning

Machine learning is changing the world

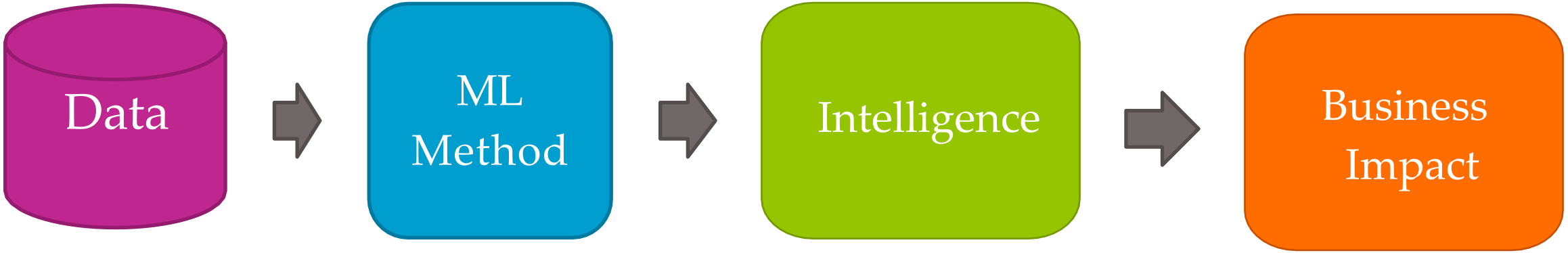
The Past of Machine Learning



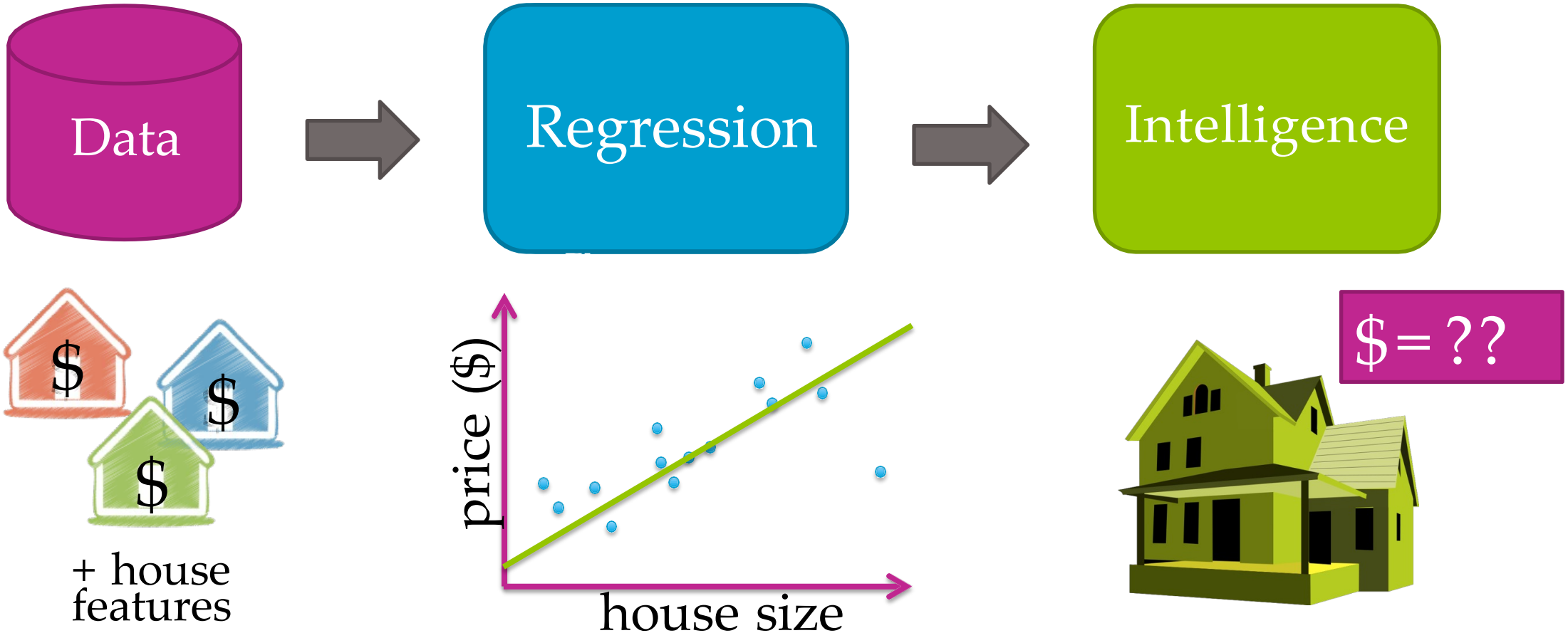
Intelligent Applications



The Present of Machine Learning



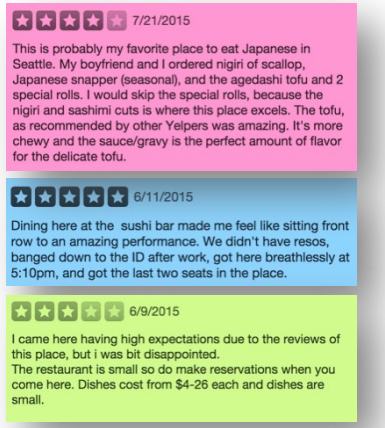
Case Study 1: Predicting house prices

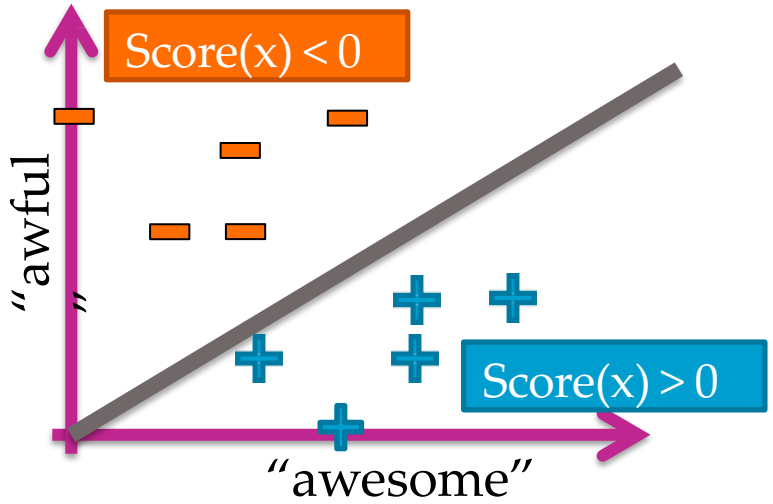


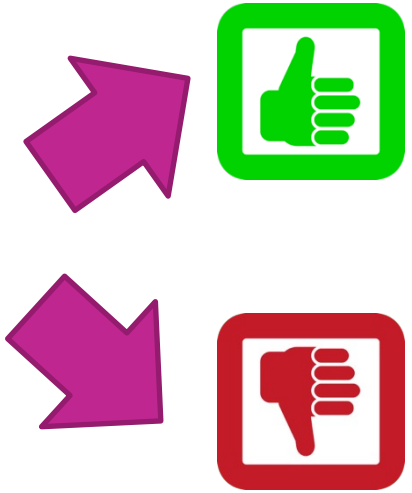
Case Study 2: Sentiment analysis

Sushi was awesome, the food was awesome, but the service was awful.

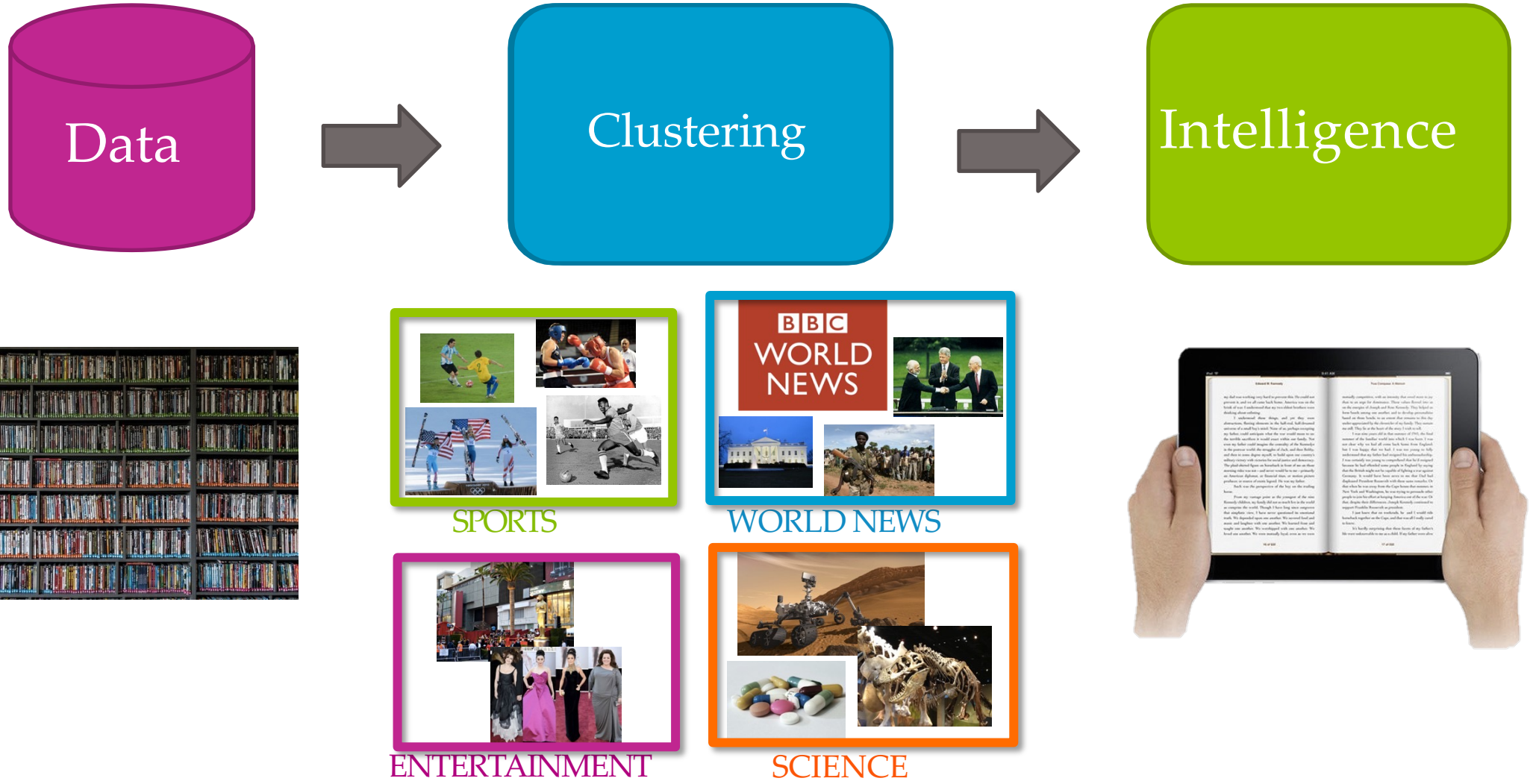
All reviews:







Case Study 3: Document retrieval



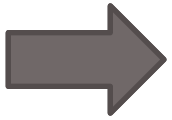
Case Study 4: Product recommendation

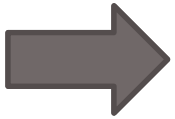
Your past purchases:

+ purchase histories of all customers

Recommended items:

Case Study 4: Product recommendation

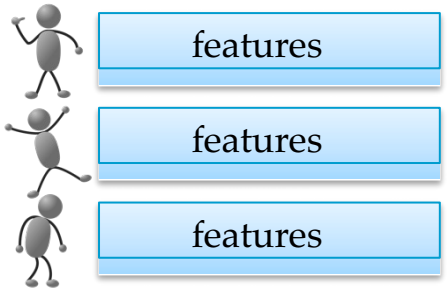




Your past purchases:

+ purchase histories of all customers

Customers

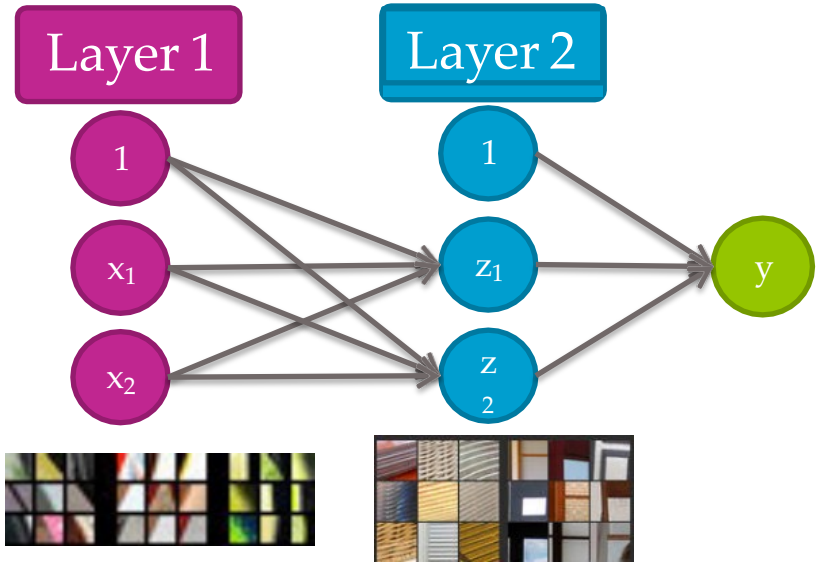


Products

Recommended items:

Case Study 5:Visual product recommender

Input images:



Nearest neighbors:

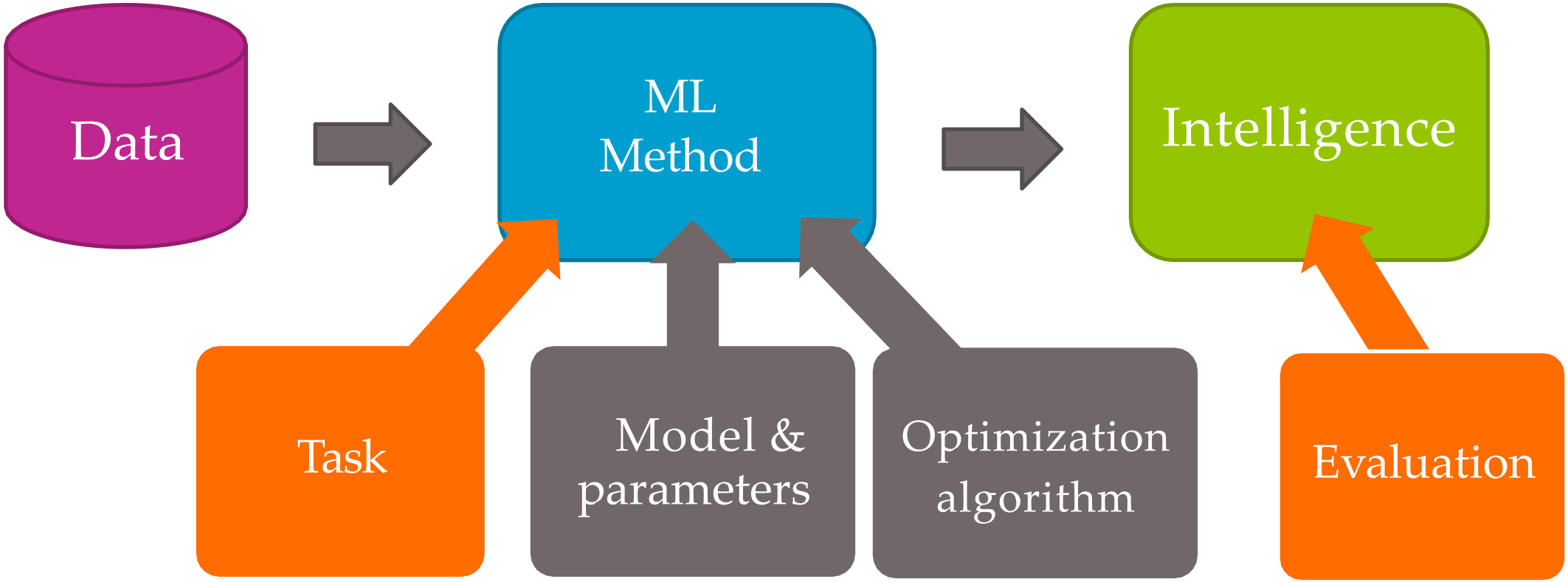
Case Study Approach

Not like other ML courses out there…

From use cases to models & algorithms

Case Study Approach

Our course is about building, evaluating and deploying *intelligence in each case study…*



Section 2 Python Basics

Why Python?

- Python is a widely used, versatile programming language ideal for business analytics and data science applications.
- Easy to start working with, making it accessible for business professionals and analysts without extensive programming backgrounds.
- Powerful data analysis and statistical computation capabilities similar to R and SAS, with extensive libraries for business intelligence.
- Supported by major machine learning frameworks such as scikit-learn, pandas, and NumPy, which are essential for business analytics workflows.

Jupyter Notebook / Google Colab

- Jupyter Notebook
  - A Jupyter notebook lets you write and execute Python code locally in your web browser
  - Interactive, code re-execution, result storage, can interleave text, equations, and images
  - Can add conda environments to Jupyter notebook
- Google Colab
  - https://colab.research.google.com/
  - Google's hosted Jupyter notebook service, runs in the cloud, requires no setup to use, provides free access to computing resources including GPUs
  - Comes with many Python libraries pre-installed

Common Operations

```
x = 10                          # Declaring two integer variables
y = 3                           # Comments start with hash
x + y           >> 13           # Arithmetic operations
x ** y          >> 1000         # Exponentiation
x / y           >> 3            # Dividing two integers
x / float(y)    >> 3.333…       # Type casting for float division
str(x) + "+"    >> "10 + 3"     # Casting integer as string and
+ str(y)                        string concatenation
```

Built-in Values

```
True, False                 # Usual true/false values

None                        # Represents the absence of something

x = None                    # Variables can be assigned None

array = [1, 2, None]        # Lists can contain None

def func():                 # Functions can return None

    return None
```

Built-in Values

```
and                         # Boolean operators in Python written
                            as plain English
or

not


if [] != [None]:            # Comparison operators == and !=
                            check for equality/inequality, return
    print("Not equal")      true/false values
```

Collections: List

Lists are **mutable arrays**.

```
names = ['Zach', 'Jay']
names[0] == 'Zach'
names.append('Richard')
print(len(names) == 3) >> True
print(names) >> ['Zach', 'Jay', 'Richard']
names += ['Abi', 'Kevin']
print(names) >> ['Zach', 'Jay', 'Richard', 'Abi', 'Kevin']
names = [] # Creates an empty list
names = list() # Also creates an empty list
stuff = [1, ['hi','bye'], -0.12, None] # Can mix types
```

List Slicing

List elements can be accessed in convenient ways.
Basic format: `some_list[start_index:end_index]`

```
numbers = [0, 1, 2, 3, 4, 5, 6]
numbers[0:3] == numbers[:3] == [0, 1, 2]
numbers[5:] == numbers[5:7] == [5, 6]
numbers[:] == numbers == [0, 1, 2, 3, 4, 5, 6]
numbers[-1] == 6 # Negative index wraps around
numbers[-3:] == [4, 5, 6]
numbers[3:-2] == [3, 4] # Can mix and match
```

Collections: Tuples

Tuples are **immutable arrays**.

```
names = ('Zach', 'Jay') # Note the parentheses
names[0] == 'Zach'
print(len(names) == 2) >> True
print(names) >> ('Zach', 'Jay')
names[0] = 'Richard' >> TypeError: 'tuple' object does not
support item assignment
empty = tuple() # Empty tuple
single = (10,) # Single-element tuple. Comma matters!
```

Collections: Dictionary

Dictionaries are **hash maps**.

```
phonebook = {} # Empty dictionary
phonebook = dict() # Also creates an empty dictionary
phonebook = {'Zach': '12-37'} # Dictionary with one item
phonebook['Jay'] = '34-23' # Add another item
print('Zach' in phonebook) >> True
print('Kevin' in phonebook) >> False
print(phonebook['Jay']) >> '34-23'
del phonebook['Zach'] # Delete an item
print(phonebook) >> {'Jay' : '34-23'}
```

Loops

To iterate over a list

```
names = ['Zach', 'Jay', 'Richard']
for name in :
    print('Hi ' + name + '!')
```

```
>> Hi Zach!
   Hi Jay!
   Hi Richard!
```

To iterate over indices and values

```
# One way
for i in range(len(names)):
    print(i, names[i])

# A different way
for i, name in enumerate(names):
    print(i, name)
```

```
>> 1 Zach
   2 Jay
   3 Richard
```

Loops

To iterate over a dictionary

```
phonebook = {'Zach': '12-37', 'Jay': '34-23'}
for name in phonebook:                          >> Jay
    print(name)                                    Zach

for number in phonebook.values():               >> 12-37
    print(number)                                  34-23

for name, number in phonebook.items():          >> Zach 12-37
    print(name, number)                            Jay 34-23
```

Note: Whether dictionary iteration order is guaranteed depends on the version of Python.

Classes

```
class Animal(object):
    def __init__(self, species, age):
        self.species = species
        self.age = age

    def is_person(self):
        return self.species

    def age_one_year(self):
        self.age += 1

class Dog(Animal):
    def age_one_year(self):
        self.age += 7
```

```
# Constructor `a =
Animal('human', 10)`
# Refer to instance with `self`
# Instance variables are public

# Invoked with `a.is_person()`

# Inherits Animal's methods
# Override for dog years
```

Installing Packages in Python

pip installs Python packages, conda installs packages which may contain software written in any language

Issues may arise when using pip and conda together. It is best practice to first use conda to install as many packages as possible and use pip to install remaining packages after. [1]

```
conda install -n myenv [package_name][=optional version number]
```

Install packages using pip in a conda environment (necessary when package not available through conda)

```
conda install -n myenv pip                    # Install pip in environment
conda activate                                # Activate environment #
myenv pip install [package_name][==optional version number]   Install package

pip install -r <requirements.txt>            # Install packages from file
```

[1] https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html#using-pip-in-an-environment

Installing Packages in Python

Importing Package Modules

```
# Import ‘os’ and ‘time’ modules
import os, time

# Import under an alias
import numpy as np
np.dot(x, y)              # Access components with pkg.fn

# Import specific submodules/functions
from numpy import linalg as la, dot as matrix_multiply
# Can result in namespace collisions...
```

Tutorial

Tutorial

1. Install Anaconda referring to the practice materials if you want to use your own PC
   - *Installing Anaconda on Windows*
   - *Installing Anaconda on MacOS*

2. Install the following packages:
   - *NumPy*
   - *Pandas*
   - *Matplotlib*
   - *Seaborn*
   - *Statsmodels*
   - *SciKit-Learn*